

***** Runtime Engineers focus on engine and game itself while tools programmers work on the file format loaders and cool stuff. Senior engineers are sometimes asked to take leading roles meaning while coding they get to make scheduling decisions and technically direct the team. Technical directors watch over the project to inform of possible challenges, possible new technological approaches in the market. CTO or chief technical officer is as high ranking as engineering positions tend to get in a game team. It technically directs the entire studio.

***** The **game engine** could be regarded as both the runtime facility and the toolset used to program and organize different aspects of a game. Regardless of how, all game engines ultimately deal and have to overcome very same concerns in very same aspects, leading to the creation of logical components that are called by generic names such as collision detector, etc. Due to the closeness of the interaction and ties of animations and physics, two generic approaches for their organization could be taken:

- 1- To traditionally have animators do the majority of the work and provide motion and then have the physics use those animations at runtime (**The animation based physics** scenario I saw in DL2)
- 2- **Physics based animations** (such as the case in Euphoria)

Cross platform engines have to consider an Independent layer that implements and serves the basic measuring functionalities to them. This usually happens by taking advantage of the already existing function prototypes and defining them for different platforms, the case of Steam's Proton pretty much.

Core Systems are basic software utilities needed and used within software, the following are some of them:

- **Assertions:** Lines of error checking code for catching errors(usually stripped out of the final production build.
- **Memory Management:** Game engines implement their own custom memory allocation systems to ensure high speed allocations.
- **Math library:** Games are math intensive hence math libraries are needed.
- **Custom data structures and Algorithms:** If uninterested in using existing libraries such as STL, the implementation of data structures such as linked lists, dynamic arrays , binary trees , hash maps and algorithms such as search and sort are required.

Resources are another essential component of game engines, typically accessed by resource managers provides an interface for accessing and modifying existing assets, either in UE centralized consistent approach or ad hoc approaches.

Rendering Engines are usually designed in the following layered architecture:

- 1- **The low-level renderer:** Which focuses on rendering primitives,texts,fonts and changing GPU state via dynamic lighting system and material system as quickly as possible without much regard for which parts will be visible or not. The initialization of the available graphics devices and setting them up for usage by graphics APIs in this layer need a reasonable amount of code

that's handled by a component called graphics **device interface**. *Computer Graphics: Principles and Practice in C* and *3D Computer Graphics, Third Edition* are good books on lighting and shaders.

- 2- **Scene Graph/Culling Optimizations** is a layer added to make up for the lack of regard for geometry visibility in the previous layer as that layer draws anything. It's in this very stage that methods such as **frustum cull**(removing objects that camera cannot see) and the more advanced **spatial subdivision** data structure are used to improve rendering efficiency by allowing the potentially visible set (PVS) of objects to be determined very quickly. The general rule is that the scene graph must be a separate module from the base renderer to allow for flexibility.
- 3- **Visual effects**: These include :
 - **Particle Systems** (for smoke, fire, water splashes) and Decal Systems (for bullet holes , foot prints , etc) tend to be handled by an effects system component that manages their specialized rendering needs.
 - **Light mapping and environment mapping** which are usually handled internally within the renderer
 - **Dynamic shadows**
 - **Post Effects** (including blur,HDR,Anti Aliasing) which take affect as an integral part of the renderer or as operators on it's output buffers.
- 4- **Front End**: 2D graphics overlay in the form of HUD, Menu ,GUI (inventory overlay,Console), FMVs (**full motion videos** recorded with game's rendering engine or another rendering package PRE RECORDED ISH old

cutscene style) and IGC(**in game cinematics** whether in Fallout or Resident evil form).

- 5- **Profiling & Debugging**: The facility for tracking performance and memory usage and all.
- 6- **Collision and Physics**: These are two intertwined systems allowing for world interactivity to be a thing. **Rigid body dynamics** in particular allows for objects to be affected by force and torques. Many devs tend to either stick to Havok or use PhysX for collision detection and physics. ODE is a good open source physics and collision engine.
- 7- **Animation**: There are 5 basic types of animations used in games.
 - sprite / texture animation
 - rigid body hierarchy animation
 - skeletal animation
 - vertex animation
 - morph targets

Skeletal animation system achieves the 3D animating thru imposing animation on 3D mesh by using relatively simple bone system and posing, it is the most common animation system in games today. The support for these systems is obviously implemented within the very renderer which gets the poses as palettes of matrices. The process of renderer transforming each vertex by the matrix in the palette to generate the final blended vertex position (Basically the process of loading the vertexes from matrix palette) is called *skinning*.

It's important to note that the implementation of physics also happens according to the skeletal animation system, specifically during the *ragdolls*. While physics system determines the positions and orientations of the various parts of the body by treating them as a constrained system of rigid bodies. The animation system calculates the palette of matrices required by the rendering engine in order to draw the character on-screen.

- 8- **HID (Human Interface Devices):** Essentially the support for possible custom human I/O device is implemented in this HID section, which itself could be implemented in 2 levels:
 - A- A low level interface for setting HOW to handle the devices capabilities and functionality (setting things such as dead zones, etc).
 - B- A higher level interface for the customizing the mapping of devices for the games.
- 9- **Audio:**For Direct X platforms Microsoft provides an excellent audio toolsuite called XACT supported at runtime by their future rich XAudio 2 and X3DAudio APIs.

10- Online Multiplayer/Networking:

Multiplayer/networked games have to function, render and handle things based on a new set of deciding factors related to networking, demanding additional preparations within the engine which are better implemented from the start.

11- Gameplay Foundation Systems:

The rules of the game assets and the effects of their behaviors' and interactions on each other , player mechanics and the like are typically implemented in the engine's native language or some high level language

or both using a categorization of objects that are dynamic and ones that are static... usually in an object oriented manner for obvious reasons. A software object model is used as template for the implementation. Part of this system is an event system implemented by sending message objects to be received by functions called event handlers.

12-

Company History

Natural Motion

Product history

Endorphin

It's first product was Endorphin is a Maya plug-in that permits animators to run full biomechanical simulations on characters and export the resulting animations as if they had been hand animated. The biomechanical model accounts for center of gravity, the character's weight distribution, and detailed knowledge of how a real human balances and moves under the influence of gravity and other forces.

Euphoria

Its second product, Euphoria, is a real-time version of Endorphin intended to produce physically and biomechanically accurate character motion at run time under the influence of unpredictable forces.